

Universidad Católica del Norte

Escuela de Ingeniería de Coquimbo

Laboratorio 01

Detección exploratoria de patrones de discurso en textos de redes sociales y medios digitales

Asignatura: Machine Learning

Profesor: Dr. Juan Bekios Calfa

Ayudantes: Sr. Victor Jopia y Sr. Andres Armijo

Fecha de entrega: 23 de abril de 2026

Coquimbo, Chile
Abril de 2026

1. Contexto del laboratorio

Este laboratorio está pensado para estudiantes de pregrado que comienzan a trabajar con recolección y análisis de textos desde distintas fuentes digitales. La actividad propone construir un *pipeline* simple, claro y reproducible que permita capturar publicaciones y noticias, unificarlas en una estructura tabular común y dejar los datos listos para análisis posteriores.

El notebook actualizado se centra en cuatro ideas principales. Primero, obtener textos desde la API de X. Segundo, recuperar noticias desde fuentes RSS, especialmente Radio Cooperativa y de manera opcional La Tercera. Tercero, consolidar todo en un **DataFrame** con una estructura mínima común. Cuarto, guardar ese corpus en un formato binario eficiente y aplicar transformaciones básicas para un análisis exploratorio inicial.

La orientación didáctica del notebook no consiste solo en ejecutar código, sino en comprender qué hace cada parte del proceso. Por esa razón, el material explica cada bloque antes de su ejecución, describe la utilidad práctica de cada paso y muestra ejemplos simples de inspección de resultados.

2. Propósito y objetivos

2.1. Propósito general

Guiar al estudiante en la construcción de un flujo básico de adquisición, consolidación, pre-procesamiento y almacenamiento de textos provenientes de X, RSS y una fuente opcional adicional, utilizando Python en Google Colab.

2.2. Objetivos específicos

Al finalizar la actividad, el estudiante debería ser capaz de:

1. Conectarse a la API de X mediante un *Bearer Token*.
2. Leer noticias desde un *feed* RSS y convertirlas en filas de un **DataFrame**.
3. Consolidar textos de múltiples fuentes en una sola tabla.
4. Aplicar una limpieza básica de texto en español.

5. Obtener frecuencias de palabras, representar el corpus con TF-IDF y realizar una agrupación simple con K-Means.
6. Visualizar el corpus en dos dimensiones mediante PCA.
7. Guardar el corpus en un archivo binario eficiente y, adicionalmente, almacenarlo en MongoDB usando una estrategia de *upsert*.

3. Estructura mínima del conjunto de datos

El laboratorio exige una tabla con al menos las siguientes columnas:

Columna	Descripción
id	identificador del registro
fuelle	origen del texto (X, RSS, TikTok, etc.)
texto	contenido textual principal

En el notebook actualizado se agregan además columnas complementarias que resultan útiles para trazabilidad y análisis:

Columna	Uso
fecha	fecha o instante de publicación
url	enlace asociado al registro
autor	autor o cuenta de origen
consulta	consulta usada en la extracción desde X
texto_limpio	versión preprocesada del texto

4. Mapa del notebook

El cuaderno se organiza de forma secuencial. La idea es ejecutar cada celda en orden y observar la salida antes de avanzar. El flujo general es el siguiente:

1. Instalación de librerías.
2. Importación de módulos.

3. Configuración general y credenciales.
4. Recolección de datos desde X.
5. Lectura de noticias desde Radio Cooperativa RSS.
6. Lectura opcional de otras fuentes, como La Tercera o TikTok desde un archivo CSV.
7. Construcción del **DataFrame** consolidado.
8. Guardado del corpus en formato binario.
9. Limpieza y preprocesamiento.
10. Frecuencia de palabras.
11. Representación TF-IDF.
12. Clustering simple con K-Means.
13. Reducción de dimensionalidad con PCA.
14. Almacenamiento del corpus en MongoDB.
15. Verificación final y preguntas guía.

5. Arquitectura general del laboratorio

Para que el estudiante no vea el cuaderno como una secuencia aislada de celdas, es útil interpretar el laboratorio como una arquitectura sencilla de procesamiento de datos textuales. Esta arquitectura organiza el trabajo en módulos lógicos que pueden reutilizarse y ampliarse en laboratorios posteriores.

5.1. Capas principales

La arquitectura propuesta se puede entender en cinco capas:

1. **Capa de adquisición:** obtiene datos desde X, RSS y, opcionalmente, desde un archivo CSV con textos de TikTok.

2. **Capa de normalización:** transforma cada fuente a un mismo esquema tabular con columnas homogéneas.
3. **Capa de persistencia local:** guarda el corpus en formatos binarios como Parquet y Pickle para reutilizarlo sin repetir la extracción.
4. **Capa de análisis:** aplica limpieza de texto, frecuencias, TF-IDF, clustering y visualización con PCA.
5. **Capa de persistencia externa:** almacena los documentos en MongoDB mediante operaciones de *upsert*.

5.2. Flujo general de datos

Fuentes de texto → DataFrames parciales → corpus consolidado →
almacenamiento binario → preprocesamiento → análisis → MongoDB

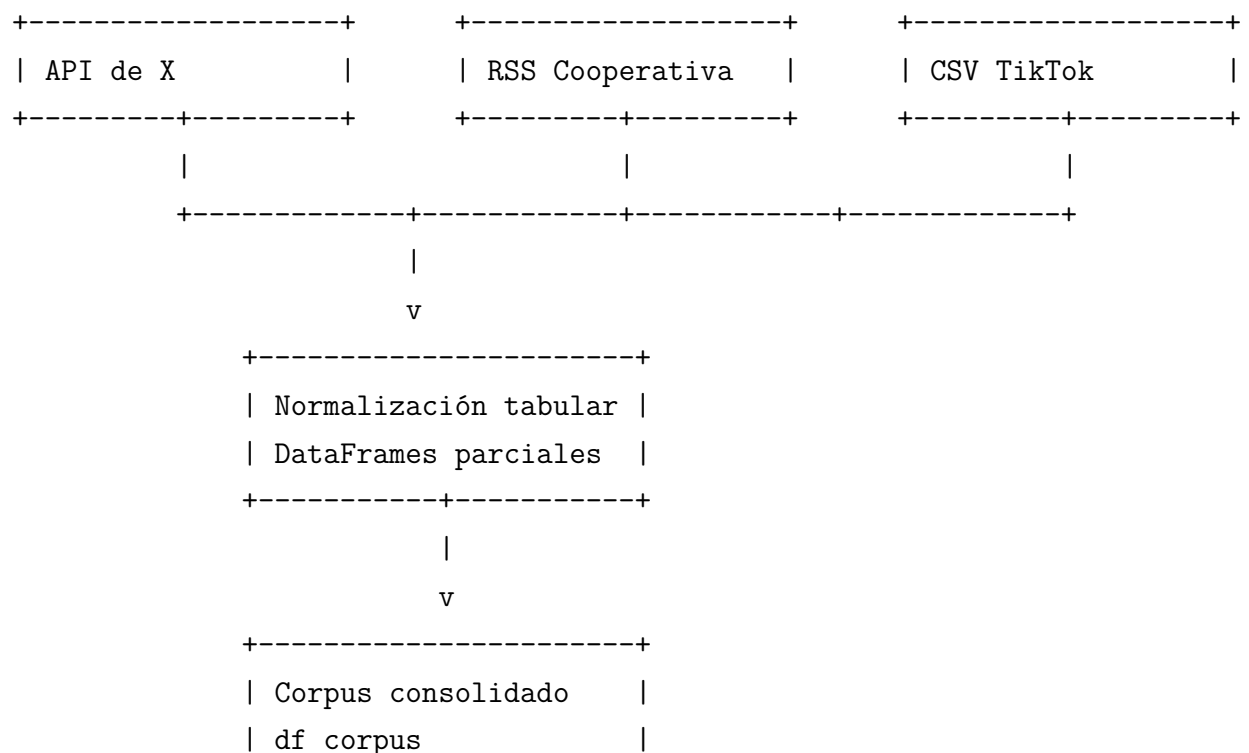
Este flujo muestra que el laboratorio no solo recolecta textos, sino que construye una pequeña tubería de datos reproducible. Cada bloque entrega una salida que sirve de entrada para el siguiente.

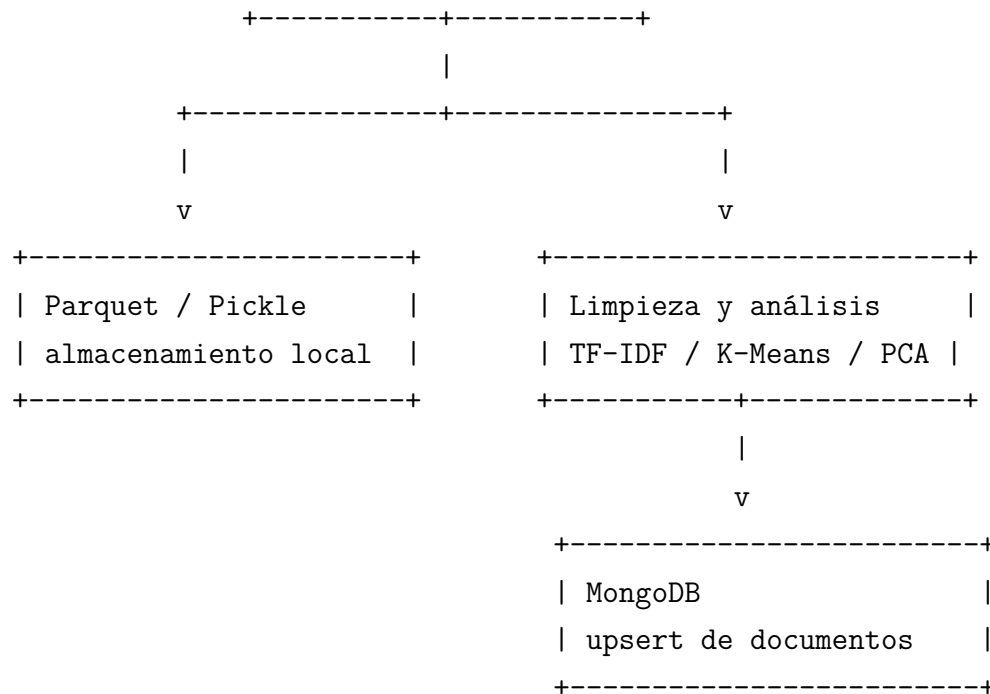
5.3. Módulos funcionales sugeridos

Aunque el notebook está escrito como un cuaderno secuencial, conceptualmente puede dividirse en los siguientes módulos:

Módulo	Responsabilidad principal
<code>configuracion</code>	Define consultas, URLs, nombres de archivos, credenciales y parámetros generales.
<code>fuentes_x</code>	Consulta la API de X y devuelve publicaciones en forma tabular.
<code>fuentes_rss</code>	Lee RSS y transforma noticias en una estructura homogénea.
<code>fuentes_csv</code>	Carga una fuente opcional desde archivo CSV y la adapta al corpus.
<code>consolidacion</code>	Une los <code>DataFrame</code> parciales y valida columnas mínimas.
<code>persistencia_binaria</code>	Guarda y recupera el corpus en Parquet y Pickle.
<code>preprocesamiento</code>	Limpia texto, elimina ruido y genera la columna <code>texto_limpio</code> .
<code>analisis</code>	Calcula frecuencias, TF-IDF, clusters y proyecciones PCA.
<code>persistencia_nosql</code>	Inserta o actualiza documentos en MongoDB.

5.4. Representación esquemática





5.5. Valor didáctico de la arquitectura

Presentar el laboratorio con esta arquitectura ayuda a que el estudiante entienda que cada parte cumple una función concreta dentro de un sistema de analítica de datos. Además, facilita futuras extensiones, por ejemplo incorporar nuevas fuentes, cambiar el método de vectorización o reemplazar K-Means por otro algoritmo sin rehacer todo el flujo.

6. Librerías y entorno de trabajo

El notebook se diseñó para ser ejecutado en Google Colab. Entre las librerías más relevantes se encuentran:

- `pandas`, para manipulación de tablas.
- `requests`, para llamar a la API de X.
- `feedparser`, para leer RSS.
- `pyarrow`, para guardar archivos Parquet.
- `nltk`, para stopwords en español.

- `scikit-learn`, para TF-IDF, K-Means y PCA.
- `pymongo`, para persistencia en MongoDB.

Un ejemplo resumido de instalación e importación inicial es el siguiente:

```
!pip install pandas requests feedparser matplotlib scikit-learn nltk pymongo
pyarrow

import re
import json
import pandas as pd
import requests
import feedparser
import matplotlib.pyplot as plt

from collections import Counter
from datetime import datetime, timezone
from getpass import getpass
from pymongo import MongoClient, UpdateOne
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from nltk.corpus import stopwords
```

7. Configuración general y credenciales

Una parte importante del notebook consiste en dejar todos los parámetros relevantes en una sola sección. Esto mejora la claridad y facilita la reutilización del código. En el notebook actualizado, además de la consulta para X y las direcciones RSS, se incluyen explícitamente el nombre de la base de datos y el nombre de la colección de MongoDB, de modo que la configuración quede consistente con las celdas de almacenamiento y verificación.

```
# Consulta general sobre universidades o tecnologia
X_QUERY = "universidad OR educacion OR tecnologia lang:es -is:retweet"
X_MAX_RESULTS = 500

COOPERATIVA_RSS_URL = "https://www.cooperativa.cl/noticias/site/tax/port/all/
rss_3___1.xml"
```

```
COOPERATIVA_MAX_ITEMS = 25

LATERCERA_RSS_URL = "https://www.latercera.com/arc/outboundfeeds/rss/?outputType=
    xml"
LATERCERA_MAX_ITEMS = 25

MONGO_DB_NAME = "LaboratorioML"
MONGO_COLLECTION_NAME = "corpus_textos"

PARQUET_OUTPUT_NAME = "corpus_consolidado_laboratorio_01.parquet"
PICKLE_OUTPUT_NAME = "corpus_consolidado_laboratorio_01.pkl"
```

Las credenciales se solicitan de manera segura mediante `getpass`, evitando dejarlas escritas en el cuaderno:

```
X_BEARER_TOKEN = getpass("Ingresa tu Bearer Token de X (o presiona Enter para
    omitir): ")
MONGODB_URI = getpass("Ingresa la URI de MongoDB (o presiona Enter para omitir):
    ")
```

8. Extracción de textos desde X

La primera fuente trabajada en el laboratorio es X. El notebook define una función para encapsular la conexión a la API, ejecutar una consulta y devolver el resultado como un `DataFrame`. Desde el punto de vista pedagógico, esta decisión permite separar tres responsabilidades: la autenticación, la consulta y la transformación del resultado.

La lógica general de la función es la siguiente:

- construir el encabezado de autenticación con el *Bearer Token*;
- enviar una consulta con parámetros como `query` y `max_results`;
- extraer campos útiles del JSON de respuesta;
- convertir esos campos a una tabla homogénea.

```
def buscar_posts_x(query, bearer_token, max_results=10):
    """
```

```
Busca publicaciones recientes en X y devuelve un DataFrame.  
"""  
  
# Construcción del request  
# Llamada a la API  
# Extracción de campos relevantes  
# Retorno como DataFrame estandarizado
```

Luego se ejecuta la consulta y el resultado se guarda en `df_x`. El cuaderno también enseña a persistir esta tabla individualmente en formato Parquet y volver a cargarla, lo que resulta útil para no repetir llamadas a la API en cada sesión.

9. Lectura de noticias desde RSS

La segunda fuente principal del laboratorio es RSS. Aquí se utiliza `feedparser` para leer un *feed* y convertir sus entradas en una tabla. Se propone como fuente obligatoria el RSS de Radio Cooperativa y como fuente adicional opcional el RSS de La Tercera.

```
def leer_rss(url_rss, nombre_fuente, max_items=20):  
    """  
    Lee un RSS y devuelve un DataFrame con estructura estandar.  
    """  
  
    feed = feedparser.parse(url_rss)  
    registros = []  
    # Título, resumen, enlace y fecha  
    return pd.DataFrame(registros)
```

La idea pedagógica es que el estudiante observe que, aunque X y RSS son fuentes distintas, ambas pueden terminar representadas con un mismo esquema tabular. Esa homogeneización es clave para trabajar luego con un solo corpus.

10. Fuente opcional adicional: TikTok desde archivo CSV

El notebook deja una alternativa opcional para incorporar textos de TikTok mediante un archivo CSV previamente preparado. Esta decisión es útil en entornos docentes, porque el

acceso directo a plataformas externas puede ser más complejo o depender de credenciales, herramientas o restricciones adicionales.

La función de apoyo busca columnas típicas como `texto`, `caption` o `descripcion`, y transforma el archivo en una estructura compatible con el corpus general.

11. Construcción del corpus consolidado

Una vez extraídos los textos desde las fuentes disponibles, el notebook los concatena en una sola tabla llamada `df_corpus`. Además, elimina filas sin contenido útil y reinicia el índice para asegurar consistencia.

```
df_corpus = pd.concat(  
    [df_x, df_tiktok, df_cooperativa, df_latercera],  
    ignore_index=True  
)  
  
df_corpus = df_corpus.dropna(subset=["texto"]).reset_index(drop=True)
```

En este punto del flujo se recomienda verificar:

- cantidad de documentos por fuente;
- nombres de columnas disponibles;
- presencia de valores nulos;
- consistencia general del contenido textual.

Esta revisión temprana es importante porque evita seguir con un corpus mal formado.

12. Almacenamiento binario del corpus

Una novedad relevante del notebook actualizado es el énfasis en almacenamiento binario eficiente. En lugar de depender solo de CSV, el laboratorio propone guardar el corpus consolidado en formatos como Parquet y, de manera complementaria, Pickle.

12.1. Por qué usar formatos binarios

El documento explica que CSV es simple y universal, pero no siempre es la mejor opción cuando el conjunto de datos crece o cuando se requiere preservar mejor tipos de datos. Parquet permite compresión y lectura eficiente por columnas, mientras que Pickle facilita guardar estructuras de Python de forma directa.

```
df_corpus.to_parquet(
    PARQUET_OUTPUT_NAME,
    index=False,
    compression="snappy"
)

df_corpus.to_pickle(PICKLE_OUTPUT_NAME)
```

Después del guardado, el notebook realiza una lectura de verificación para comprobar que el archivo puede recuperarse correctamente.

13. Preprocesamiento básico de texto

Antes de aplicar técnicas de minería de texto, el cuaderno realiza una limpieza básica. El objetivo es reducir ruido y dejar los documentos en una forma más útil para el análisis.

Las transformaciones principales son:

- convertir a minúsculas;
- eliminar enlaces;
- eliminar menciones y signos innecesarios;
- remover caracteres no deseados;
- eliminar *stopwords* en español.

```
STOPWORDS_ES = set(stopwords.words("spanish"))

def limpiar_texto(texto):
    texto = texto.lower()
    texto = re.sub(r"http\S+", " ", texto)
```

```
texto = re.sub(r"@w+", " ", texto)
texto = re.sub(r"#", " ", texto)
texto = re.sub(r"[^a-zA-Záéíóúñü\s]", " ", texto)
tokens = [t for t in texto.split() if t not in STOPWORDS_ES]
return " ".join(tokens)
```

Una característica didáctica útil del notebook es que muestra un ejemplo antes y después de la limpieza, de forma que el estudiante pueda visualizar concretamente el efecto del preprocesamiento.

14. Frecuencia de palabras

La primera técnica exploratoria aplicada al corpus es el conteo de frecuencias. La idea es detectar qué vocabulario domina globalmente y qué términos aparecen con mayor presencia por fuente.

```
def obtener_frecuencias(textos):
    todas = " ".join(textos).split()
    return Counter(todas)

frecuencias_globales = obtener_frecuencias(df_corpus["texto_limpio"])
```

Esta parte del laboratorio también incluye un gráfico de barras con las palabras más frecuentes, lo que permite discutir si ciertos términos reflejan tendencias temáticas, agendas mediáticas o rasgos discursivos particulares de una fuente.

15. Representación TF-IDF

La segunda etapa analítica del notebook consiste en convertir los textos en una matriz numérica mediante TF-IDF. Desde una perspectiva introductoria, esto ayuda a explicar cómo un texto puede pasar a una representación vectorial adecuada para análisis de similitud y aprendizaje no supervisado.

```
vectorizer = TfidfVectorizer(max_features=1000)
X_tfidf = vectorizer.fit_transform(df_corpus["texto_limpio"])
```

Posteriormente, el cuaderno calcula pesos promedio por término para ofrecer una interpretación básica de cuáles palabras poseen mayor relevancia media dentro del corpus.

16. Clustering simple con K-Means

Una vez construida la matriz TF-IDF, el notebook aplica K-Means para agrupar documentos por similitud léxica. La intención no es obtener una segmentación definitiva, sino introducir una primera herramienta de agrupamiento.

```
k = 4 if len(df_corpus) >= 4 else max(1, len(df_corpus))

modelo_kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

df_corpus["cluster"] = modelo_kmeans.fit_predict(X_tfidf)
```

Luego se inspeccionan los términos más representativos de cada *cluster*, lo que permite interpretar los grupos a partir de las palabras de mayor peso en sus centroides.

17. Visualización con PCA

Como la representación TF-IDF puede tener muchas dimensiones, el notebook utiliza PCA para proyectar el corpus a dos dimensiones y facilitar su visualización. Se construyen dos gráficos: uno coloreado por fuente y otro coloreado por *cluster*.

```
X_dense = X_tfidf.toarray()
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_dense)

df_corpus["pca_1"] = X_pca[:, 0]
df_corpus["pca_2"] = X_pca[:, 1]
```

Esta visualización sirve para discutir si las fuentes tienden a separarse, si ciertos grupos se mezclan o si aparece alguna estructura discursiva incipiente.

18. Persistencia en MongoDB

El notebook actualizado incorpora una etapa de almacenamiento en MongoDB. Se utiliza una función que guarda el `DataFrame` mediante operaciones *upsert*, es decir, actualizando documentos existentes o creando nuevos cuando sea necesario.

```
def guardar_en_mongodb(df, mongo_uri, db_name, collection_name):  
    """  
    Guarda un DataFrame en MongoDB usando upsert por _id.  
    """  
    if not mongo_uri:  
        print("No se ingreso URI de MongoDB.")  
        return
```

La llamada principal utiliza las variables de configuración que ahora sí aparecen explícitamente en el notebook:

```
guardar_en_mongodb(  
    df=df_corpus,  
    mongo_uri=MONGODB_URI,  
    db_name=MONGO_DB_NAME,  
    collection_name=MONGO_COLLECTION_NAME  
)
```

Finalmente, una celda de verificación comprueba la conexión y muestra algunos registros, lo que permite cerrar el flujo completo de captura, transformación y almacenamiento.

19. Preguntas guía para la discusión

El notebook termina con preguntas orientadas a la interpretación de resultados. Estas preguntas pueden incorporarse directamente en la guía del laboratorio o utilizarse como base para la discusión en clases:

1. ¿Qué palabras aparecen con mayor frecuencia en todo el corpus?
2. ¿Qué diferencias léxicas se observan entre las fuentes?
3. ¿Los *clusters* encontrados parecen coherentes con los temas presentes?

4. ¿El gráfico PCA sugiere separación clara entre fuentes o grupos?
5. ¿Qué ventajas ofrece Parquet frente a CSV en este laboratorio?
6. ¿Qué aporta MongoDB cuando el corpus crece y necesita persistencia flexible?

20. Sugerencias metodológicas para los estudiantes

Para desarrollar bien el laboratorio, se recomienda que el estudiante:

- ejecute el notebook en orden;
- revise la salida de cada celda antes de continuar;
- documente cualquier problema de conexión o autenticación;
- compare el texto original con el texto limpio;
- interprete los resultados más allá de la ejecución mecánica del código.

También es recomendable que, al redactar su informe, el estudiante describa no solo qué hizo, sino por qué lo hizo, qué salida obtuvo y qué limitaciones observó en el corpus recuperado.

21. Conclusión

El notebook actualizado del Laboratorio 01 constituye una guía introductoria, pero completa, para construir un pequeño flujo de minería de textos con múltiples fuentes. Su mayor fortaleza pedagógica es que integra en una sola experiencia varios conceptos relevantes: adquisición de datos, consolidación tabular, almacenamiento binario, limpieza textual, representación vectorial, agrupamiento, visualización y persistencia en una base NoSQL.

La actualización realizada mejora la coherencia de la configuración al dejar explícitos el nombre de la base de datos y de la colección de MongoDB en la sección de parámetros. Con ello, el material queda mejor alineado con sus propias celdas explicativas y con el flujo de trabajo que el estudiante debe seguir.

A. Resumen breve del flujo del laboratorio

1. Se instalan e importan las librerías necesarias.
2. Se configuran consultas, URLs, credenciales y nombres de archivos.
3. Se recuperan textos desde X y desde RSS.
4. Se incorpora de manera opcional una fuente adicional desde CSV.
5. Se consolida toda la información en un solo **DataFrame**.
6. Se guarda el corpus en Parquet y Pickle.
7. Se limpia el texto y se generan variables derivadas.
8. Se analizan frecuencias, TF-IDF, clusters y PCA.
9. Se almacena el corpus en MongoDB con *upsert*.
10. Se verifican los resultados y se discuten hallazgos iniciales.